

1480. Running Sum of 1d Array

Easy Topics Companies Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0],...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

Solve this question in a real interview before? 1/6

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for (int i = 1; i < nums.length; i++) {
4             nums[i] = nums[i] + nums[i - 1];
5         }
6         return nums;
7     }
8 }
```

Saved

In 4, Co

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected

[1,3,6,10]

27. Remove Element

Easy Topics Companies Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return the number of elements in `nums` which are not equal to `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`. To get accepted, you need to do the following things:

Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.

Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                             // It is sorted with no values equaling val.
```

```
int k = removeElement(nums, val); // Calls your implementation
```

```
assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [3,2,2,3]`, `val = 3`

Output: `2`, `nums = [2,2,_,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `2`. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0;
4         for (int i = 0; i < nums.length; i++) {
5             if (nums[i] != val) {
6                 nums[k] = nums[i];
7                 k++;
8             }
9         }
10        return k;
11    }
12 }
13 }
```

Save

Ln 10, Col 18

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

`nums =`
`[3,2,2,3]`

`val =`
`3`

Output

`[2,2]`

Expected

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of unique elements in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`
Output: 2, `nums = [1,2,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively.
It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`
Output: 5, `nums = [0,1,2,3,4,_,_,_,_,_]`
Explanation: Your function should return `k = 5`, with the first five elements of `nums` being 0, 1, 2, 3, and 4. There are four underscores left in the array, which can be filled with arbitrary values.

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k = 1;
4         for (int i = 1; i < nums.length; i++) {
5             if (nums[i] != nums[i-1]) {
6                 nums[k] = nums[i];
7                 k++;
8             }
9         }
10        return k;
11    }
12 }
13 }
```

Saved

In 10, Col 18

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums =`
`[1,1,2]`

Output

`[1,2]`

Expected

`[1,2]`

53. Maximum Subarray

[return](#) [Topics](#) [Companies](#)

Given an integer array `nums`, find the **subarray** with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`

Output: 1

Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`

Output: 23

Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

$1 \leq \text{nums.length} \leq 10^5$

$-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Java Auto

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int currentSum = nums[0];
4         int maxSum = nums[0];
5         for (int i = 1; i < nums.length; i++) {
6             currentSum = Math.max(nums[i], currentSum + nums[i]);
7             maxSum = Math.max(maxSum, currentSum);
8         }
9         return maxSum;
10    }
11 }
12 }
13 }
```

Saved

Ln 9; Co

Testcase Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

`nums =`
`[-2,1,-3,4,-1,2,1,-5,4]`

Output

6

Expected

6

GitHubShuffle the Array - LeetCodeChatGPT

leetcode.com/problems/shuffle-the-array/

Problem List<>>Submit

Ask GeminiVerify it's youPremium

DescriptionEditorialSolutionsSubmissions

1470. Shuffle the Array

EasyTopicsCompaniesHint

Given the array `nums` consisting of `2n` elements in the form `[x1, x2, ..., xn, y1, y2, ..., yn]`.

Return the array in the form `[x1, y1, x2, y2, ..., xn, yn]`.

Example 1:

Input: `nums = [2,5,1,3,4,7], n = 3`
Output: `[2,3,5,4,1,7]`
Explanation: Since `x1=2, x2=5, x3=1, y1=3, y2=4, y3=7` then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1], n = 4`
Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2], n = 2`
Output: `[1,2,1,2]`

Constraints:

- `1 <= n <= 500`
- `nums.length == 2n`
- `1 <= nums[i] <= 109`

Discover moreComputer Education

6.5K16662 Online

Code

JavaAuto

```
1 class Solution {
2     public int[] shuffle(int[] nums, int n) {
3         int[] ans = new int[2 * n];
4         int index = 0;
5
6         for (int i = 0; i < n; i++) {
7             ans[index++] = nums[i];
8             ans[index++] = nums[i + n];
9         }
10
11         return ans;
12     }
13 }
```

SavedLn 1, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

nums =

[2,5,1,3,4,7]

n =

3

Output

[2,3,5,4,1,7]

Expected

[2,3,5,4,1,7]

Windows taskbar icons: File Explorer, Google Chrome, Microsoft Edge, VS Code, and others.

System tray: ENG IN, 11:35, 07-08-2026

1732. Find the Highest Altitude

Easy Topics Companies Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length `n` where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all $(0 \leq i < n)$. Return the **highest altitude** of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`
Output: 1
Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`
Output: 0
Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

Discover more Search Engines

Seen this question in a real interview before? 1/6

3.5K 427 50 Online

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int altitude = 0;
4         int maxAltitude = 0;
5
6         for (int g : gain) {
7             altitude += g;
8             maxAltitude = Math.max(maxAltitude, altitude);
9         }
10
11         return maxAltitude;
12     }
13 }
```

Saved

Ln 13, Col 2

Accepted Runtime: 0 ms

Case 1 Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1

Contribute a testcase

Description Editorial Solutions Submissions

49. Group Anagrams

Medium Topics Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

22.4K 425 1 1 1

472 Online

Code

Java Auto

```
1 class Solution {
2     public List<List<String>> groupAnagrams(String[] strs) {
3         Map<String, List<String>> map = new HashMap<>();
4
5         for (String str : strs) {
6             char[] chars = str.toCharArray();
7             Arrays.sort(chars);
8             String key = new String(chars);
9
10            map.putIfAbsent(key, new ArrayList<>());
11            map.get(key).add(str);
12        }
13
14        return new ArrayList<>(map.values());
15    }
16 }
```

Saved

Ln 16, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`strs =`
`["eat","tea","tan","ate","nat","bat"]`

Output

`[["eat","tea","ate"],["bat"],["tan","nat"]]`

Expected

`[["bat"],["nat","tan"],["ate","eat","tea"]]`

Contribute a testcase

347. Top K Frequent Elements

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where `n` is the array's size.

```
1 class Solution {
2     public int[] topKFrequent(int[] nums, int k) {
3         // Step 1: Count frequencies
4         Map<Integer, Integer> freqMap = new HashMap<>();
5         for (int num : nums) {
6             freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
7         }
8
9         // Step 2: Create buckets
10        List<Integer>[] bucket = new ArrayList[nums.length + 1];
11
12        for (int key : freqMap.keySet()) {
13            int freq = freqMap.get(key);
14            if (bucket[freq] == null) {
15                bucket[freq] = new ArrayList<>();
16            }
17            bucket[freq].add(key);
18        }
19
20        // Step 3: Collect top k elements
21        int[] result = new int[k];
22        int index = 0;
23
24        for (int i = bucket.length - 1; i >= 0 && index < k; i--) {
25            if (bucket[i] != null) {
26                for (int num : bucket[i]) {
27                    result[index++] = num;
28                    if (index == k) {
29                        break;
30                    }
31                }
32            }
33        }
34
35        return result;
36    }
37 }
```

Accepted Runtime: 0 ms